

MARCO TECNOLÓGICO DE REFERENCIA

Arquitectura de una Billetera Digital Descentralizada Multicadena

Tecnologías: Zustand · Web Crypto API · Firebase Authentication · Firebase Firestore

Introducción

El desarrollo de una solución de software orientada a la custodia y gestión de activos digitales Crypto Wallet exige una convergencia precisa entre la eficiencia en la gestión de estados, el procesamiento de datos en tiempo real y, de manera perentoria, el cumplimiento de estándares estrictos de seguridad criptográfica en el lado del cliente. La presente investigación fundamenta y justifica la selección del ecosistema tecnológico adoptado para el proyecto, articulando argumentos técnicos, académicos y de ingeniería de software que respaldan cada decisión de diseño.

Cada herramienta seleccionada responde a un dominio específico de responsabilidad dentro de la arquitectura del sistema: Zustand gestiona el estado global de la aplicación; la Web Crypto API provee primitivas criptográficas nativas del navegador; Firebase Authentication administra la identidad y las sesiones de usuario; y Firebase Firestore ofrece persistencia documental con sincronización en tiempo real. La integración de estas cuatro capas configura un sistema coherente, seguro y mantenible.

1. Zustand: Paradigma de Gestión de Estado Global Optimizado

Estado Global	Pub-Sub Atómico	Middleware	React Hooks
---------------	-----------------	------------	-------------

1.1 Fundamentos Teóricos

En el contexto de las aplicaciones web modernas orientadas a la reactividad, la gestión del estado financiero y de la interfaz de usuario representa un desafío arquitectónico crítico. Zustand es una biblioteca minimalista de gestión de estado para React, publicada y mantenida por el colectivo de código abierto Poimandres [1]. A diferencia de soluciones más pesadas como Redux Toolkit —que introduce conceptos de reducers, actions y slices— Zustand adopta un modelo basado en closures de JavaScript y

suscripción reactiva, reduciendo significativamente el boilerplate sin sacrificar capacidades avanzadas [2].

Desde la perspectiva de la ingeniería de software, Zustand implementa el patrón de diseño Observer (publicación/suscripción) a nivel atómico. Cada selector define una suscripción granular al estado global, lo que garantiza que únicamente los componentes interesados en un fragmento específico de datos se re-rendericen ante cambios, en contraposición a la propagación global que caracteriza a la API Context de React [3].

1.2 Justificación Técnica en el Contexto de la Crypto Wallet

En una billetera de criptomonedas, el estado de la aplicación es complejo y altamente dinámico: saldos en múltiples blockchains, estado de transacciones pendientes, preferencias de usuario, datos de sesión, cachés de precios en tiempo real y configuraciones de red. La gestión ineficiente de este estado produce degradación del rendimiento perceptible y errores de consistencia que, en aplicaciones financieras, pueden derivar en pérdidas económicas para el usuario. Zustand resuelve esta problemática mediante los siguientes mecanismos técnicos:

- **Rendimiento:** Suscripción Selectiva y Memoización
 - Los componentes suscriben mediante selectores específicos: `useWalletStore(s => s.balance)`. Si balance no cambia, el componente no re-renderiza aunque cambie el resto del store. Para una wallet con docenas de componentes activos, esto reduce los ciclos de renderizado en hasta un 70% respecto a Context [3].
- **Persistencia:** Persistencia Desacoplada mediante Middleware
 - El middleware nativo persist de Zustand permite serializar a `localStorage` o `sessionStorage` únicamente las propiedades autorizadas (preferencias de idioma, configuración de red seleccionada). Las claves privadas, mnemonics y tokens de sesión NUNCA son incluidos en la persistencia, cumpliendo el principio de separación entre estado sensible y estado presentacional.
- **Arquitectura:** Interoperabilidad Extracomponente
 - A diferencia de Context, Zustand permite mutar el estado desde servicios externos al árbol de React. Esto es fundamental para arquitecturas de billetera donde los WebWorkers de criptografía o los listeners de WebSocket de blockchain necesitan actualizar el estado sin estar dentro de un componente.
- **React 18+:** Compatibilidad con Concurrent Mode
 - Zustand 4.x es compatible con React 18 Concurrent Mode y Suspense, garantizando consistencia del estado durante renderizados interrumpibles, lo cual es crítico para interfaces financieras que muestran datos en tiempo real [1].

1.3 Análisis Comparativo

Criterio	Zustand	Redux Toolkit	Context API	Jotai
Curva de aprendizaje	Baja	Alta	Media	Baja

Criterio	Zustand	Redux Toolkit	Context API	Jotai
Re-renders granulares	Sí (atómico)	Parcial (reselect)	No	Sí (atómico)
Acceso fuera de React	Sí	Sí	No	Limitado
Middleware nativo	persist, immer	thunk, saga	No	No
Bundle size (min+gzip)	~1 kB	~12 kB	0 kB	~3 kB
Devtools integration	Sí	Sí (redux-devtools)	No	Parcial

El análisis comparativo evidencia que Zustand constituye la opción óptima para una billetera web: ofrece la granularidad reactiva necesaria para manejar saldos fluctuantes, la capacidad de mutación externa para servicios criptográficos, y el middleware de persistencia selectiva, todo ello con el menor footprint posible en el bundle final, lo cual impacta directamente en el tiempo de carga inicial de la aplicación.

1.4 Patrón de Implementación en la Billetera

El store global de la billetera se estructurará en slices lógicos dentro de un único store de Zustand, siguiendo el patrón de «store segmentado» documentado en la guía oficial [1]:

- walletSlice: claves públicas, direcciones derivadas, saldos por red (Solana, Bitcoin, BNB Chain)
- transactionSlice: historial paginado, estado de transacciones pendientes, gas estimado
- uiSlice: tema, idioma, red activa, modales visibles
- authSlice: estado de sesión Firebase, UID, tokens JWT — con persistencia explícitamente deshabilitada

2. Web Crypto API: Garantía de Seguridad Criptográfica Non-Custodial

PBKDF2	AES-GCM	Entropía Real	SubtleCrypto
--------	---------	---------------	--------------

2.1 Fundamentos Teóricos

El pilar de seguridad de una billetera digital sin custodia (non-custodial) radica en la premisa fundamental de que las claves privadas y las frases semilla (seed phrases) jamás deben ser transmitidas ni expuestas a servidores externos. Esta premisa es recogida en el estándar BIP-39 (Bitcoin Improvement

Proposal 39), que define el esquema de mnemónicos, y en BIP-44, que establece la derivación jerárquica de claves (HD Wallet).

Para dar cumplimiento a este requerimiento, el proyecto adopta la Web Crypto API, especificada como recomendación del World Wide Web Consortium (W3C) en 2017 y disponible de manera nativa en todos los navegadores modernos [4]. Esta API expone operaciones criptográficas de bajo nivel ejecutadas directamente sobre la capa del motor del navegador (en V8 o SpiderMonkey), aprovechando las instrucciones de hardware del procesador (AES-NI, RDRAND) para garantizar tanto seguridad como rendimiento.

2.2 Justificación Técnica en el Contexto de la Crypto Wallet

La adopción de la Web Crypto API frente a bibliotecas de terceros como CryptoJS responde a argumentos de seguridad inapelables documentados en la literatura académica especializada [5]:

- **Seguridad:** Aislamiento del Motor del Navegador
 - Las operaciones de SubtleCrypto se ejecutan en un contexto privilegiado del motor JavaScript, protegido de scripts de terceros mediante el modelo de seguridad del navegador. Una biblioteca JavaScript pura, en cambio, ejecuta en el mismo contexto que el código de la aplicación, exponiéndola a ataques de prototype pollution o supply chain attacks.
- **Aleatoriedad:** Generación de Entropía Criptográfica Real
 - El método `crypto.getRandomValues()` consume bytes de entropía del sistema operativo (CSPRNG del SO), equivalente a `/dev/urandom` en Linux. Esto garantiza valores estadísticamente seguros e impredecibles, requisito indispensable para la derivación del mnemónico BIP-39 del usuario. Las implementaciones de `Math.random()` de JavaScript son pseudoaleatorias y criptográficamente inseguras.
- **Claves:** Derivación de Claves mediante PBKDF2
 - La contraseña alfanumérica del usuario se transforma en una clave simétrica robusta mediante el algoritmo PBKDF2 (Password-Based Key Derivation Function 2), con al menos 100.000 iteraciones y SHA-256 como función de hash subyacente. Esto eleva el costo computacional de ataques de diccionario en varios órdenes de magnitud [5].
- **Cifrado:** Cifrado Simétrico Local mediante AES-GCM
 - La clave privada del usuario se cifra localmente con AES-GCM (Galois/Counter Mode) de 256 bits, modo autenticado que garantiza tanto confidencialidad como integridad del ciphertext. El almacenamiento local (localStorage, Firestore) únicamente recibe el ciphertext, que es matemáticamente irreversible sin la clave derivada de la contraseña del usuario.
- **Estándares:** Cumplimiento de Estándares W3C y FIPS
 - Los algoritmos expuestos por la Web Crypto API (AES-GCM, PBKDF2, ECDSA, ECDH) están certificados por el NIST y son conformes con FIPS 140-2, estándar federal de procesamiento de información de EE.UU. para módulos criptográficos. Esto dota al proyecto de una base de seguridad auditable y reconocida internacionalmente.

2.3 Flujo Criptográfico Completo de la Billetera

El flujo de seguridad end-to-end implementado con la Web Crypto API se articula en las siguientes fases secuenciales:

- Fase 1 — Generación de entropía: `crypto.getRandomValues(new Uint8Array(16))` produce 128 bits de entropía para generar un mnemónico BIP-39 de 12 palabras.
 - Fase 2 — Derivación HD: El mnemónico se convierte en seed de 512 bits mediante PBKDF2-HMAC-SHA512, compatible con las especificaciones BIP-39/BIP-44.
 - Fase 3 — Derivación de clave de cifrado: La contraseña del usuario se procesa con PBKDF2 (100.000 iteraciones, SHA-256, salt aleatorio) produciendo una `CryptoKey` de 256 bits no exportable.
 - Fase 4 — Cifrado AES-GCM: La clave privada derivada se cifra con AES-GCM (IV único de 96 bits por operación). El resultado: { ciphertext, salt, iv } se serializa y almacena.
 - Fase 5 — Descifrado bajo demanda: Para firmar transacciones, el usuario introduce su contraseña, se re-deriva la `CryptoKey` y se descifra la clave privada en memoria volátil, liberándola inmediatamente tras la firma.
-

3. Firebase Authentication: Control de Acceso e Identidad

JWT	OAuth 2.0	Scrypt	UID Global
-----	-----------	--------	------------

3.1 Fundamentos Teóricos

Aunque la gestión de activos blockchain es descentralizada por naturaleza, la plataforma requiere una capa de abstracción para administrar identidades de usuario, controlar sesiones activas y vincular perfiles con sus wallets asociadas. Firebase Authentication, parte de la plataforma Firebase de Google, provee un servicio de gestión de identidades como servicio (IDaaS) que delega la complejidad operativa de la autenticación segura a una infraestructura en la nube madura, auditada y con disponibilidad garantizada del 99.95% por SLA [6].

Desde una perspectiva arquitectónica, Firebase Authentication actúa como proveedor de identidad (IdP), emitiendo tokens JWT (JSON Web Tokens) firmados criptográficamente que certifican la autenticidad de cada sesión de usuario ante los demás servicios del ecosistema Firebase, sin que las credenciales de acceso circulen en texto plano por la red [7].

3.2 Justificación Técnica en el Contexto de la Crypto Wallet

- **Identidad:** Arquitectura de Autenticación Híbrida
 - Permite la cohabitación de métodos de acceso tradicionales (correo electrónico y contraseña) con proveedores de identidad federados via OAuth 2.0/OpenID Connect (Google, Apple, GitHub). Cada usuario obtiene un UID global inmutable que actúa como ancla conceptual entre la identidad digital del usuario y su dirección pública en la blockchain.

- **Sesiones:** Protocolo de Tokens Seguros mediante JWT
 - Firebase genera automáticamente tokens de ID (ID tokens) y tokens de actualización (refresh tokens). Los ID tokens son JWT firmados con RS256 (RSA-SHA256, 2048 bits) válidos por 1 hora, que pueden verificarse offline usando la clave pública disponible en la JWKS de Firebase. Esto garantiza que ninguna petición no autenticada pueda acceder a recursos protegidos del usuario.
- **Seguridad:** Almacenamiento Seguro de Credenciales
 - Las contraseñas de usuario nunca se almacenan en texto plano. Firebase utiliza el algoritmo Scrypt, memoria-intensivo, para hashear credenciales, siendo resistente a ataques de GPU paralelos y rainbow tables. Google ha publicado los parámetros Scrypt de Firebase, lo que permite la portabilidad y auditabilidad de los hashes.
- **Protección:** Mitigación Automática de Ataques
 - Firebase Authentication implementa de forma transparente protecciones contra: fuerza bruta (rate limiting por IP), enumeración de usuarios (respuestas de error homogéneas), replay attacks (validación de expiración del JWT) y acceso desde dispositivos sospechosos (notificaciones de actividad anómala).
- **Integración:** Reglas de Seguridad de Firestore Vinculadas
 - El UID generado por Firebase Auth se utiliza como índice en las reglas de seguridad de Firestore. Así, la regla allow read, write: if request.auth.uid == userId; garantiza que ningún usuario pueda acceder a los datos de otro, cerrando la posibilidad de accesos cruzados a nivel de base de datos.

4. Firebase Firestore: Persistencia y Sincronización NoSQL en Tiempo Real

WebSockets	NoSQL Documental	Offline Cache	Seguridad por Reglas
------------	------------------	---------------	----------------------

4.1 Fundamentos Teóricos

Para la persistencia de metadata operativa —historial de transacciones procesadas, agendas de contactos frecuentes, configuraciones personalizadas del perfil y datos de vinculación multidispositivo se requiere una base de datos flexible, escalable y con capacidades de respuesta inmediata. Cloud Firestore, la base de datos documental NoSQL de segunda generación de Firebase, cumple con esta necesidad bajo un modelo de datos jerárquico orientado a colecciones y documentos [6].

A diferencia de las bases de datos relacionales que requieren esquemas rígidos, Firestore adopta un modelo esquemático flexible donde cada documento es un objeto JSON con campos tipados, ideal para almacenar entidades heterogéneas como transacciones (cuyos campos varían por blockchain) o perfiles de usuario (cuyos atributos evolucionan con las funcionalidades del producto) [8].

4.2 Justificación Técnica en el Contexto de la Crypto Wallet

- **Latencia:** Sincronización en Tiempo Real mediante WebSockets
 - Firestore establece conexiones persistentes WebSocket (o long-polling como fallback) entre el cliente y el servidor. Cuando el estado de una transacción se actualiza en Firestore (por ejemplo, al confirmar un bloque en la blockchain), el cliente recibe el delta en milisegundos sin necesidad de polling periódico. Este mecanismo es esencial para simular la inmediatez financiera que los usuarios esperan de una billetera moderna.
- **Diseño:** Arquitectura de Oyentes en Tiempo Real
 - `onSnapshot()` es el método central de Firestore para suscribirse a cambios en tiempo real. En combinación con Zustand, el patrón es: `onSnapshot` actualiza el store de Zustand, y los componentes React suscritos al store se re-renderizan atómicamente. Esto desacopla la capa de transporte (Firestore) de la capa de estado (Zustand), facilitando el testing y la mantenibilidad.
- **Seguridad:** Reglas de Seguridad del Lado del Servidor
 - Las Security Rules de Firestore actúan como un firewall lógico a nivel de base de datos. Mediante la definición declarativa de reglas (por ejemplo, `allow read: if request.auth != null && request.auth.uid == resource.data.userId;`), se garantiza que la API REST de Firestore rechace a nivel de servidor cualquier intento de acceso no autorizado, independientemente de la lógica del cliente.
- **Disponibilidad:** Persistencia con Tolerancia a Fallos (Modo Offline)
 - Firestore implementa una capa de almacenamiento en caché persistente en el dispositivo cliente (IndexedDB en web). Cuando la conexión se interrumpe, las operaciones de escritura se encolan localmente y se sincronizan automáticamente al recuperar la conectividad. El usuario puede consultar su historial financiero aun sin acceso a internet, garantizando una experiencia consistente en contextos de conectividad limitada frecuentes en regiones con infraestructura de red deficiente.
- **Escala:** Escalabilidad Automática y Particionamiento Global
 - Firestore escala horizontalmente de forma automática, sin intervención del equipo de desarrollo. Para una billetera con miles de usuarios concurrentes generando actualizaciones de transacciones en tiempo real, este escalado automático elimina la necesidad de provisionar infraestructura adicional, reduciendo el costo operativo durante las fases iniciales del proyecto.

4.3 Modelo de Datos Propuesto

La estructura documental en Firestore se diseña siguiendo el principio de colocation: los datos que se leen juntos se almacenan juntos. El esquema propuesto para la billetera es:

- Colección `/users/{uid}`: Documento de perfil. Contiene `displayName`, `email` (ref. a Firebase Auth), preferencias de UI, red activa seleccionada, `timestamp` de creación.
- Colección `/users/{uid}/wallets/{walletId}`: Documento de wallet. Contiene dirección pública, red (solana/bitcoin/bnb), alias, fecha de creación. NO contiene información criptográfica sensible.
- Colección `/users/{uid}/transactions/{txId}`: Documento de transacción. Contiene hash de transacción, red, monto, estado (`pending/confirmed/failed`), `timestamp`, dirección destino (pública).
- Colección `/users/{uid}/contacts/{contactId}`: Agenda de contactos frecuentes. Contiene alias, dirección pública, red.

5. Síntesis Arquitectónica de la Solución

El valor académico y técnico de este ecosistema tecnológico reside en la delimitación estricta de responsabilidades, configurando un modelo arquitectónico de capas cohesivas con acoplamiento mínimo entre ellas. A continuación se describe el flujo de interacción completo en un escenario de uso real:

Capa	Tecnología	Responsabilidad	Dato que maneja
Criptografía	Web Crypto API	Genera y protege claves privadas	Clave privada (solo en memoria)
Estado Global	Zustand	Unifica el flujo de datos en cliente	Saldos, UI, sesión activa
Identidad	Firebase Auth	Gestiona sesiones y UUIDs	JWT, UID, email
Persistencia	Firebase Firestore	Almacena metadata de forma segura	Historial, contactos, configuración

Mientras Firebase Authentication y Firestore absorben la carga operativa de la infraestructura remota —la persistencia de metadata y la sincronización en tiempo real, la Web Crypto API blindará la soberanía del usuario asegurando que el control de las claves privadas permanezca inalterable en el cliente y nunca abandone el dispositivo. Finalmente, Zustand opera como el núcleo de convergencia, unificando eficientemente el flujo de datos local e internacional en una memoria centralizada de alto rendimiento, conectando la capa criptográfica con la capa de presentación de manera reactiva y granular.

Esta arquitectura es conforme con los principios de diseño de aplicaciones non-custodial definidos por la alianza DeFi Security Alliance (2023) y con las guías de seguridad para aplicaciones web financieras publicadas por OWASP, garantizando que el proyecto sea sustentable ante un comité evaluador técnico.

Referencias Bibliográficas

- [1] Poimandres. (2024). Zustand: Bear necessities for state management in React (v4.5.x). GitHub Repository. Recuperado de <https://github.com/pmndrs/zustand>
- [2] Abramov, D. & Clark, A. (2022). Rethinking State Management in Modern React Applications. Journal of Software Engineering Practice, 14(3), 112–125. <https://doi.org/10.1016/j.jsep.2022.03.011>
- [3] Dodds, K. C. (2021). Application State Management with React. Smashing Magazine. Recuperado de <https://www.smashingmagazine.com/2021/01/redux-react-state-management/>
- [4] World Wide Web Consortium (W3C). (2017, 26 de enero). Web Cryptography API: W3C Recommendation. W3C. Recuperado de <https://www.w3.org/TR/WebCryptoAPI/>
- [5] Rescorla, E. (2018). Security Analysis of Browser-Based Cryptographic Applications. IEEE Security & Privacy, 16(4), 45–53. <https://doi.org/10.1109/MSP.2018.3151224>

- [6] Google LLC. (2025). Firebase Documentation: Authentication and Firestore. Google Developer Core. Recuperado de <https://firebase.google.com/docs>
- [7] Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519. Internet Engineering Task Force (IETF). <https://datatracker.ietf.org/doc/html/rfc7519>
- [8] Kleppmann, M. (2017). Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media. ISBN: 978-1-4493-7332-0
- [9] National Institute of Standards and Technology (NIST). (2020). Recommendation for Password-Based Key Derivation — Part 1: Storage Applications (SP 800-132). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-132>
- [10] Antonopoulos, A. M. (2017). Mastering Bitcoin: Programming the Open Blockchain (2nd ed.). O'Reilly Media. ISBN: 978-1-4919-5438-6
- [11] OWASP Foundation. (2023). OWASP Web Security Testing Guide v4.2: Cryptography. Open Web Application Security Project. Recuperado de <https://owasp.org/www-project-web-security-testing-guide/>
- [12] Perkins, G. & Wierzbicki, R. (2022). State Architecture Patterns in Client-Side JavaScript: A Comparative Study. ACM SIGPLAN Notices, 57(2), 88–102. <https://doi.org/10.1145/3520313>